GUÍA DE TALLER PRÁCTICO / EVALUACIÓN PRÁCTICA

Taller: Frontend con Next.js, Clean Architecture, WebSockets y Soporte Dual REST/GraphQL

FACULTAD BARBERI DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS
DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS INTELIGENTES

1 Presentación y Arquitectura del Sistema

El presente taller práctico tiene como objetivo desarrollar en equipos de trabajo la aplicación frontend empresarial utilizando **Next.js** (App Router y TypeScript), consumiendo e integrando los servicios desarrollados en los módulos de **NestJS**. En esta entrega, los estudiantes estructurarán el sistema bajo **Clean Architecture** (*Feature-First*), incorporando comunicación reactiva en tiempo real mediante **WebSockets** y demostrando el desacoplamiento arquitectónico mediante la refactorización de un módulo con soporte de datos dual intercambiable entre **REST** y **GraphQL**.

2 Especificación de Requerimientos del Sistema

Los equipos de trabajo deberán implementar rigurosamente los siguientes bloques funcionales y técnicos:

1. Arquitectura Limpia (Clean Architecture) Feature-First

Organizar el proyecto por módulos de funcionalidad (*Feature-First*) siguiendo el diseño visto en clase:

- **Dominio (`domain/`):** Entidades puras con validación de invariantes, puertos (interfaces de repositorio) y casos de uso orquestadores, sin dependencias de React, Next.js ni librerías HTTP.
- **Infraestructura (`infrastructure/`):** Implementaciones de repositorio, DTOs de red, mappers bidireccionales y DataSources (REST, GraphQL, WebSockets).
- **Presentación (`presentation/`):** Componentes visuales puros, hooks fachada desacoplados y slices de estado en Redux.

2. Flujo Completo de Autenticación y Sesión JWT

- **Vistas dedicadas:** Interfaces adaptativas para Inicio de Sesión (`/login`) y Registro (`/register`).
- **Gestión de tokens:** Almacenar el JWT de forma segura y adjuntarlo como `Authorization: Bearer <token>` en todas las peticiones a rutas protegidas.
- **Cierre de sesión (Logout):** Purgar el token de la persistencia local, resetear el store de Redux y redirigir a inicio.

3. Autorización y Control de Acceso por Permisos y Roles (Authorities)

- **Sistema de Authorities y Permisos:** Implementar un esquema de autorización granular basado en roles y permisos (*authorities*), permitiendo verificar privilegios específicos del usuario autenticado.
- **Restricción de Rutas y Vistas:** Restringir páginas y acciones según los permisos y roles verificados en el token.
- **Gestión de Privilegios:** Proveer una interfaz para auditar y gestionar los roles y permisos de los usuarios registrados.

4. Interfaz de Usuario y Experiencia (UI/UX)

- **Componentes Modulares Reutilizables:** Construir componentes tipados en TypeScript con Tailwind CSS o Shadcn/UI (botones, tarjetas, campos con validación visual, modales y tablas).
- **Paginación y Filtrado:** Incorporar paginación (numérica o cursor) y filtros de búsqueda reactivos.

- **Validación y Feedback:** Validación de formularios robusta con notificaciones *toast*, alertas o *banners* accesibles.

5. Gestión del Estado Centralizado con Redux y Hooks Fachada

- **Store Centralizado en Redux:** Administrar el estado global exclusivamente con **Redux Toolkit** (sesión, permisos y colecciones de datos del dominio).
- **Hooks Fachada:** Los componentes de React no acceden al store ni a la red directamente; consumen *custom hooks* (ej. `useRoutines`, `useAuth`) que encapsulan los dispatchers de Redux y los casos de uso.

6. Funcionalidad en Tiempo Real con WebSockets

Incorporar al menos una funcionalidad reactiva en tiempo real (notificaciones, estados en vivo, chat o feed sincronizado):

- **DataSource Socket:** El cliente socket reside en `infrastructure/datasources/` bajo una interfaz de contrato.
- **Handshake con JWT:** Autenticar la conexión enviando el token JWT en el protocolo de enlace.
- **Sincronización Reactiva:** Despachar eventos entrantes directamente al store de Redux sin requerir recargas de página.

7. Módulo Dual: Abstracción de Datos REST y GraphQL

Refactorizar un módulo completo para operar indistintamente contra endpoints **REST** y **GraphQL**:

- **Puerto Único:** Una sola interfaz de repositorio en dominio para las operaciones requeridas.
- **DataSources Duales:** Dos implementaciones (`RestDatasource` y `GraphQLDatasource`) que retornan los mismos DTOs hacia el mapper.
- **Selección por Configuración:** La fuente de datos activa se determina mediante la variable de entorno `NEXT_PUBLIC_DATA_SOURCE=rest|graphql`.
- **Transparencia Total:** Cambiar la variable no altera Casos de Uso, Entidades, Slices de Redux ni Componentes de la interfaz.

8. Funcionalidades del Dominio de Negocio

- **Lógica Específica:** Implementar todas las historias de usuario y requerimientos funcionales particulares del proyecto grupal.
- **Persistencia Transaccional:** Integración completa de las operaciones de negocio con los servicios del backend en NestJS.

9. Pruebas Automatizadas E2E, Despliegue Cloud y Calidad de Código

- **Pruebas E2E:** Suite automatizada en Cypress cubriendo flujos de autenticación, control de acceso y tiempo real.
- **Despliegue Cloud:** Frontend activo en Vercel y backend en Railway/Render/AWS con conectividad real en producción y sin errores CORS.
- **README Técnico:** Código limpio y tipado. Documentación con guía de arranque, variables `.env.example`, enlaces cloud y comparativo REST vs. GraphQL.

3 Factores Multiplicativos

La calificación individual del proyecto estará sujeta a **dos factores multiplicativos directos**: la trazabilidad del trabajo individual en el repositorio y el desempeño individual durante la sustentación oral.

ATENCION / IMPORTANTE

Factor Multiplicativo 1: Trazabilidad en Git
Se evaluará el historial de commits en el repositorio para auditar la participación individual de cada integrante del equipo:

1. Frecuencia, consistencia y relevancia de los commits individuales a lo largo de las semanas.
2. Ausencia de commits masivos de última hora o monopolización del repositorio por un único integrante. *Una participación deficiente o asimétrica impactará negativamente la nota individual sobre la calificación grupal del proyecto.*

ATENCION / IMPORTANTE

Factor Multiplicativo 2: Sustentación y Defensa Oral Individual
Durante la sesión de revisión y sustentación del proyecto:

- **Cualquier integrante del equipo seleccionado aleatoriamente** deberá estar en capacidad de explicar, depurar y defender cualquier componente, funcionalidad, hook, caso de uso o servicio de la aplicación.
- Si un estudiante no demuestra dominio conceptual o no puede responder por el código entregado por su equipo, se le aplicará un factor multiplicativo reductor individual a su calificación final.

4 Criterios de Evaluación y Rúbrica de Aprendizaje

CRITERIO EVALUATIVO	DESCRIPCIÓN DE CUMPLIMIENTO	PONDERACIÓN
Clean Architecture y Modularidad	Separación por capas (Dominio, Infraestructura, Presentación) y puertos desacoplados.	15%
Autenticación y Sesión JWT	Login, Registro, Logout y persistencia segura de token.	10%
Autorización, Roles y Authorities	Control de acceso basado en roles y permisos (*authorities*) en vistas y acciones.	10%
Interfaz UI/UX y Componentes	Componentes modulares, paginación, filtros y validaciones visuales.	15%
Gestión de Estado con Redux	Store global centralizado con Redux Toolkit y hooks fachada.	10%
Funcionalidad WebSockets	Eventos en tiempo real con JWT y sincronización con Redux.	10%
Módulo Dual: REST y GraphQL	DataSources intercambiables por variable de entorno sin modificar dominio ni UI.	10%
Funcionalidades del Dominio	Historias de usuario del proyecto grupal conectadas con el backend NestJS.	10%
Pruebas E2E, Despliegue y README	Pruebas en Cypress, despliegue funcional en la nube y documentación técnica.	10%
Trazabilidad Git (Commits)	Historial equitativo de commits y colaboración demostrable en el repositorio.	**Factor Multiplicativo**
Sustentación Oral Individual	Dominio conceptual y defensa técnica individual del código desarrollado.	**Factor Multiplicativo**

5 Metodología de Entrega y Recepción

1. **Repositorio Oficial:** Entrega en la organización del curso en GitHub.
2. **Plazo de Entrega:** Jueves 26 de Noviembre de 2026. Commits posteriores no serán calificados sin autorización.
3. **Sustentación:** Asistencia sincrónica obligatoria; cualquier integrante del equipo será evaluado individualmente sobre cualquier módulo.